

## Contrôle continu : 1 / JAVA AVANCE

Dans une Entreprise d'Ingénierie Informatique (SII) installée à Casablanca, un Client (une Entreprise de Transport) présente un cahier de charge pour la conception et le développement d'une application basée sur la Programmation Orienté Objet notamment à base du langage Java Avancée. L'application concerne le Domaine du Transport de Marchandises. L'entreprise assure le Transport de marchandises pour différents clients. L'entreprise comporte plusieurs chauffeurs, plusieurs Cargaisons. Le modèle de travail des chauffeurs est le suivant : travailler **une journée** et se reposer **deux journées**, pour les courtes distances. Et travailler **trois journées** et se reposer **trois journées** pour les longues distances.

Objet Transport :

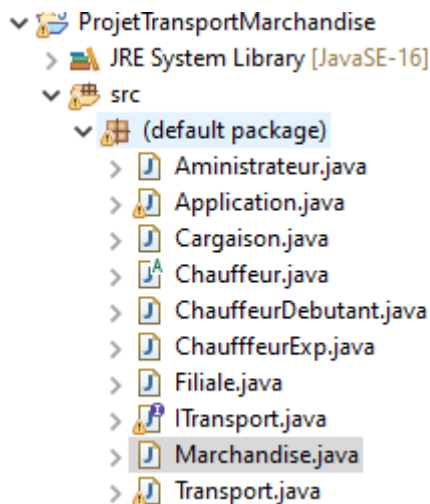
|                                                  |                      |
|--------------------------------------------------|----------------------|
| Numero Cargaison (Camion) :                      | <input type="text"/> |
| Puissance :                                      | <input type="text"/> |
| Matricle :                                       | <input type="text"/> |
| <input type="button" value="Ajouter Cargaison"/> |                      |



Dans l'entreprise il y a deux types de chauffeurs : **un chauffeur avec expérience**, c'est celui qui a une expérience de conduite qui dépasse **2 ans**, par contre **un chauffeur débutant** c'est celui qui n'a pas d'expérience en conduite. Chaque chauffeur Expérimenté est caractérisé par : un identifiant unique, un nom, une adresse, un email, un statut **isEnService** qui prend uniquement deux valeurs vrai ou faux, cette valeur représente la situation du chauffeur est ce qu'il est en Service ou en Repos dans cet instant. L'entreprise comporte également un ensemble de personnels administrateurs qui gère le système de transport en entier. Chaque administrateur est caractérisé par : un identifiant unique **numeroAdmin**, un nom, son Age, son Grade, son salaire et le service dont il appartient. Un service est caractérisé par un identifiant unique **numeroService** et le nom du service. L'entreprise comporte aussi un ensemble de types de Marchandises à transporter, des marchandises locales et des marchandises exportées depuis l'étranger. Une Marchandise est caractérisée par : un identifiant unique **numeroMarchandise**, la référence de la marchandise **refMarchandise**, et une valeur **isExporte** de type **boolean** qui désigne est ce que la marchandise est exportée depuis l'étranger ou non. L'application doit gérer l'ensemble des fonctionnalités de transports (les chauffeurs, les marchandises, les cargaisons ...). La conception et le développement de l'application doit répondre aux notions de : **diagramme de classe, agrégation, généralisation, héritage, polymorphisme, classe abstraite, Interface, redéfinition de Méthode ...**

## Questions :

1. Créer le diagramme de classe qui correspond à cette application ?
2. Implémenter les différentes classes en Java ?



3. Créer l'Interface ITransport qui contient les méthodes suivantes :
4. Enregistrer une Cargaison.

```
1 import java.util.ArrayList;
6
7 public class Transport implements ITransport{
8     List<Cargaison>    cargaisons=new ArrayList<>();
9     Map<Integer, Chauffeur>    chauffeurs=new HashMap<>() ;
10    List<Aministrateur>    administres=new ArrayList<>();
11    List<Marchandise>    marchandises=new ArrayList<Marchandise>();
12
13    @Override
14    public void enregistrerCargaison(Cargaison carg) {
15        // TODO Auto-generated method stub
16    }
17 }
```

5. Enregistrer un chauffeur en lui affectant a une cargaison.
6. Créer la liste des Cargaison de type ArrayList.
7. Créer la liste des chauffeurs de type HashMap.
8. Créer la liste des administrateurs qui gère l'application Transport de type ArrayList.
9. Créer la liste des Marchandise à Transporter de type ArrayList.
10. Créer la liste des services de type ArrayList.
11. Consulter une cargaison sachant son numéro.

```
47    @Override
48    public Cargaison consulterCargaisonSachantNumerocarg(int numerocarg) {
49        // TODO Auto-generated method stub
50    }
51 }
```

12. Consulter une cargaison par matricule.

```
35    @Override
36    public Cargaison consulterCargaisonSachantMatricule(String matricule) {
37        // TODO Auto-generated method stub
38    }
39 }
```

13. Consulter la liste de tous les cargaisons.

```

61     public List<Cargaison> consulterListedeTousLesCargaisons() {
62         // TODO Auto-generated method stub
63     }

```

14. Consulter la liste de tous les chauffeurs.

```

66     @Override
67     public Map<Integer, Chauffeur> consulterListeTousLesChauffeurs() {
68         // TODO Auto-generated method stub

```

15. ajouter une méthode calculPrime () qui calcul le prime Total des chauffeurs dans une année. Pour un chauffeur Débutant, le prime est le salaire trimestrielle, par contre pour le chauffeur Expérimenté le prime est le salaire\*10 ;

```

77     @Override
78     public double consulterPRimeTotaleDeschauffeurs() {
79         // TODO Auto-generated method stub

```

Utiliser la notion de Polymorphisme, la notion de la classe abstraite, la notion de la méthode abstraite, la notion des Interfaces, utiliser des constructeurs par défaut et des constructeurs avec paramètres, utiliser la notion du surcharge de méthode, utiliser la méthode toString() ;

Utiliser une Interface Graphique GUI en utilisant java swing ou java FX pour représenter l'application.

16. Ajouter une Fonction nombreDesChauffeursEnService ( ) qui renvoie le nombre des chauffeurs en service, on suppose que le chauffeur est en service si la valeur boolean isEnService == true.

```

109     @Override
110     public int nombreDesMarchandiseExporte() {

```

17. Ajouter une Fonction nombreDesMarchandiseExporte ( ) qui renvoie le nombre des Marchandises exportées depuis l'étranger.

```

109     @Override
110     public int nombreDesMarchandiseExporte() {

```

18. Créer une fonction IndiceChauffeurExp ( ) qui renvoie un Tableau de type Entier int [ ] qui indique les indices des chauffeurs avec Experience, on suppose dans ce cas qu'un chauffeur avec experience est celui qui a un salaire qui dépasse 8000 Dhs.

```

127     @Override
128     public int[] IndiceChauffeurExp() {
129         // TODO Auto-generated method stub
130         int [] ttemp=new int[chauffeurs.size()+1];

```

19. Créer une Fonction NumeroMatriculeDesCargaisonDesChauffeurExp ( ) qui renvoie un Tableau de type String [ ], le Tableau contient les Matricule des cargaisons des chauffeurs Expérimentés.

```

141     @Override
142     public String[] NumeroMatriculeDesCargaisonDesChauffeurExp() {
143         // TODO Auto-generated method stub
144         String [] ttempString=new String[chauffeurs.size()+1];

```